

LLM Notes

2026 Spring

Day 19: Gradient Checkpointing

CS336 Systems

主讲: Aaron

Fudan University

今日摘要

Activation checkpointing 不保存 checkpoint 区域内部所有 forward residuals, 而只保存区域边界的输入。Backward 需要内部结果时, 再重新执行该区域的 forward。它以额外计算时间换取更低的 activation memory。

在 32 个独立 XL Transformer block 的实验中, group size 为 1、2、4 时, 峰值显存分别为 30.17、33.35 和 39.69 GiB。每个 block 单独 checkpoint 最省显存。

19.1 两种 Checkpoint

需要区分两种不同的 checkpoint。

19.1.1 Training Checkpoint

训练 checkpoint 通常每隔若干 training step 保存:

- 模型参数;
- optimizer state;
- 当前 training step;
- random-number-generator state。

它用于训练中断后的恢复, 时间尺度是多个 training step。

19.1.2 Activation Checkpoint

Activation checkpoint 发生在一次 forward 和 backward 内部。

正常 forward 会保存 backward 需要的中间 tensor:

$$x \xrightarrow{f_1} h_1 \xrightarrow{f_2} h_2 \xrightarrow{f_3} h_3.$$

如果不使用 checkpoint, autograd 可能保存:

$$x, h_1, h_2, h_3.$$

使用 activation checkpoint 后, 可以只保存区域输入:

$$x.$$

Backward 需要 h_1, h_2, h_3 时, 再从 x 重新执行一次 forward。

方法	显存	计算量
不使用 checkpoint	较高	较低
使用 checkpoint	较低	较高

19.2 PyTorch Checkpoint API

PyTorch 提供:

```
from torch.utils.checkpoint import checkpoint
```

```
y = checkpoint(
    function,
    x,
    use_reentrant=False,
)
```

这里的 `function` 表示一个需要被 checkpoint 的计算区域。

例如, 对连续若干 Transformer block 进行 checkpoint:

```
def run_group(x, start=start, end=end):
    for layer_index in range(start, end):
        x = blocks[layer_index](x)
    return x
```

```
x = checkpoint(
    run_group,
    x,
    use_reentrant=False,
)
```

Forward 时主要保存 `run_group` 的输入；Backward 时重新运行该函数，以恢复区域内部所需的中间结果。

19.3 Group Size 的含义

假设模型由 N 个连续 block 构成，每 g 个 block 放入一个 checkpoint 区域：

$$\underbrace{B_0, \dots, B_{g-1}}_{\text{group 0}}, \quad \underbrace{B_g, \dots, B_{2g-1}}_{\text{group 1}}, \quad \dots$$

其中：

$$g = \text{group size.}$$

当 $g = 1$ 时，每个 block 单独构成一个 checkpoint：

$$[B_0], [B_1], [B_2], \dots$$

当 $g = 2$ 时，每两个 block 构成一个 checkpoint：

$$[B_0, B_1], [B_2, B_3], \dots$$

当 $g = 4$ 时，每四个 block 构成一个 checkpoint：

$$[B_0, B_1, B_2, B_3], \dots$$

Group 越大，需要保存的 group boundary 越少，但重新执行一个 group 时，同时存活的内部 activation 可能越多。

19.4 Checkpoint Boundary 的大小

XL block 的输入形状为：

$$(B, S, d_{\text{model}}) = (4, 2048, 2560).$$

在 FP32 下，一个 block 输入的大小为：

$$\begin{aligned} C &= B S d_{\text{model}} \times 4 \text{ bytes} \\ &= 4 \times 2048 \times 2560 \times 4 \text{ bytes} \\ &= 80 \text{ MiB.} \end{aligned}$$

此前测得，一个 block 内部为 backward 保存的 tensor logical size 约为：

$$R \approx 3651.31 \text{ MiB.}$$

虽然 logical size 不等于真实物理显存，但两者的数量级差异非常明显：

$$C \ll R.$$

因此，只保存 block 输入，并在 backward 中重新计算内部结果，可以显著降低 activation memory。

19.5 四层验证实验

首先使用四次 block 计算，并将每两个 block 放入一个 checkpoint group：

$$[B_0, B_1], \quad [B_2, B_3].$$

Saved-tensor hooks 观察到两个主要 checkpoint 输入：

$$(4, 2048, 2560), \quad (4, 2048, 2560).$$

每个 tensor 为 80 MiB，因此总 logical size 为：

$$2 \times 80 = 160 \text{ MiB.}$$

这说明 checkpoint 主要保存了两个 group 的边界输入，而没有长期保存所有 block 内部的大型 attention 和 MLP residuals。

19.6 32 个独立 Block 的实验

为了模拟真实多层模型，我们构造了 32 个独立的 Transformer block。每层拥有自己的参数，不再重复调用同一个 block。

实验测量 forward 和 backward 过程中：

`torch.cuda.max_memory_allocated()`

得到：

Group size	Peak allocated memory
1	30.17 GiB
2	33.35 GiB
4	39.69 GiB

结果表明：

$$M_{\text{peak}}(1) < M_{\text{peak}}(2) < M_{\text{peak}}(4).$$

也就是每个 block 单独 checkpoint 时，峰值显存最低。

19.7 为什么 Group Size 1 最省显存

一个粗略的显存模型是：

$$M_{\text{peak}}(g) \approx M_{\text{base}} + \frac{N}{g}C + M_{\text{inside}}(g),$$

其中：

- M_{base} 包括参数、梯度和其他长期 tensor；
- $\frac{N}{g}C$ 是 checkpoint boundary 的存储；
- $M_{\text{inside}}(g)$ 是重新执行一个 group 时的内部 activation。

Group size 增大时：

$$\frac{N}{g}C$$

会减小，但 group 内部同时存活的 activation 会增大。

在当前实验中：

$$C = 80 \text{ MiB}, \quad R \approx 3651.31 \text{ MiB}.$$

少保存几个 80 MiB boundary 所带来的收益，小于在 group 内多保留一个大型 block residual 所带来的代价，因此 group size 1 最省显存。

实测差值为：

$$33.35 - 30.17 = 3.18 \text{ GiB},$$

$$39.69 - 33.35 = 6.34 \text{ GiB}.$$

第二个差值大约是第一个的两倍，与 group 内部增加多个 block residual 的解释基本一致。

备注

这里的 peak memory 包括 32 个独立 block 的参数、backward 产生的参数梯度、checkpoint boundary 和当前计算的临时 tensor。

它不是完整 XL Language Model 的训练峰值，因为实验没有包括 embedding、LM head、loss 和 AdamW optimizer states。

19.8 什么是 Nested Checkpointing

普通分组 checkpoint 是一层结构，例如：

$$[B_0, B_1, B_2, B_3], \quad [B_4, B_5, B_6, B_7].$$

Nested checkpointing 则允许一个较大的 checkpoint 区域内部继续包含更小的 checkpoint 区域。

例如，对 8 个 block：

$$[B_0, \dots, B_7]$$

首先划分为：

$$[B_0, \dots, B_3], \quad [B_4, \dots, B_7],$$

然后每一半继续划分：

$$[B_0, B_1], \quad [B_2, B_3], \quad [B_4, B_5], \quad [B_6, B_7].$$

最后还可以继续划分到单个 block。

分析

这里的 nested 并不是说 Transformer block 并行执行，也不是模型结构变成了树。所有 block 仍然按照 $B_0 \rightarrow B_1 \rightarrow \dots$ 串行执行。

所谓嵌套，是指一个 checkpointed function 的内部，又调用了更小的 checkpointed function。

19.9 如何让 N 个 Block 的 Peak Activation Memory 最小

允许 nested checkpoint 时，可以使用 balanced recursive checkpointing：

1. 将 N 个 block 近似平均分成两半;
2. 对整个区间建立 checkpoint;
3. 在每一半内部继续递归建立 checkpoint;
4. 直到子区间只剩一个 block。

递归树的深度约为:

$$\log_2 N.$$

在 backward 的任意时刻, 只需要保留递归路径上每一级的 checkpoint boundary, 再加上当前 block 的 residual。因此 peak activation memory 近似为:

$$M_{\text{activation}} = O(C \log N + R),$$

其中:

C = 一个 checkpoint boundary 的大小,

R = 当前单个 block 执行所需的 activation memory.

忽略单个 block 的固定工作空间后, 可以简写为:

$$O(\log N).$$

代价是上层 checkpoint 的重计算会再次触发内部 checkpoint, 部分 block 可能在多个递归层级被重复执行, 因此总计算量近似为:

$$O(N \log N).$$

相比之下, 不使用 checkpoint 时, 保存所有层 activation 的空间约为:

$$O(N).$$

因此 nested checkpointing 实现了:

$O(N)$ activation memory $\longrightarrow O(\log N)$ activation memory
--

代价是:

$O(N)$ forward work $\rightarrow O(N \log N)$ recomputation work

19.10 结论

今日摘要

1. Activation checkpointing 只保存区域边界，在 backward 中重新计算内部结果；
2. 它与用于恢复训练的 training checkpoint 完全不同；
3. XL block 输入约为 80 MiB，而内部 saved tensors 约为 3651.31 MiB；
4. 32 个独立 block 的实测峰值分别为 30.17、33.35 和 39.69 GiB；
5. 当前实验中，每个 block 单独 checkpoint 最省显存；
6. Balanced nested checkpointing 可以将关于层数的 activation memory 从 $O(N)$ 降到约 $O(\log N)$ ；
7. 这一优化以额外的 $O(N \log N)$ 重计算为代价。